

Тема урока: наследование

1. Введение в наследование
2. Наследование в Python
3. Определение и переопределение атрибутов и методов
4. Неявное наследование, класс `object`
5. Многоуровневое наследование

Аннотация. Урок посвящен наследованию в Python.

Введение в наследование

Одной из базовых концепций объектно-ориентированного программирования является наследование. Наследование — это механизм получения новых классов из уже существующих. Класс, созданный путем наследования, приобретает все свойства (атрибуты) и поведение (методы) того класса, от которого был наследован.

Класс, от которого наследуется класс, называется **родительским** или **базовым классом**. Класс, который наследуется от родительского (базового) класса, называется **дочерним классом** или **наследником**.



Дочерние классы также называют **подклассами**, а родительские классы — **суперклассами**.

Мы можем рассмотреть наследование с точки зрения биологии, например, представив ребенка, наследующего определенные черты от своего родителя. Ребенок может унаследовать рост и цвет глаз родителя, также дети могут носить одну и ту же фамилию со своими родителями.

Наследование в Python

Родительские классы выступают в качестве шаблонов для дочерних классов. Родительские классы позволяют создавать дочерние классы посредством наследования без необходимости каждый раз писать один и тот же код заново. Другими словами, каждый дочерний класс может использовать методы и атрибуты родительского класса.

Например, мы можем иметь родительский класс, описывающий банковский счет, и дочерние классы, описывающие личный счет и бизнес счет. Многие методы между личными и бизнес счетами будут похожи, например, методы снятия и внесения денег, поэтому они могут принадлежать родительскому классу. Подклассы же могут дополнительно иметь собственные специфические методы.

Синтаксис наследования в Python достаточно прост. Для того чтобы создать дочерний класс, необходимо при его определении в скобках указать родительский класс:

```
class ParentClass:                                # родительский класс
    pass

class ChildClass(ParentClass):                     # дочерний класс
    pass
```

В качестве дальнейших примеров рассмотрим родительский класс `Animal`, описывающий животное, и дочерний класс `Cat`, описывающий кошку:

```
class Animal:
    def __init__(self, name, age):
        self.name = name                # имя животного
        self.age = age                  # возраст животного

    def sleep(self):
        return f'{self.name} спит zZ'

class Cat(Animal):
    pass
```

В классе `Animal` определены методы `__init__()` и `sleep()`, в то время как класс `Cat` является пустым. Но поскольку класс `Cat` является подклассом класса `Animal`, он наследует как метод `__init__()`, так и метод `sleep()`. Это означает, что экземпляры класса `Cat` могут свободно использовать данные методы.

Приведенный ниже код:

```
class Animal:
    def __init__(self, name, age):
        self.name = name
        self.age = age

    def sleep(self):
        return f'{self.name} спит zZ'

class Cat(Animal):
    pass

animal = Animal('Роджер', 2)
cat = Cat('Кемаль', 1)

print(animal.name, animal.age)
print(animal.sleep())
print(cat.name, cat.age)
print(cat.sleep())
```

ВЫВОДИТ:

```
Роджер 2
Роджер спит zZ
Кемаль 1
Кемаль спит zZ
```

Так как класс `Cat` использует метод `__init__()` класса `Animal`, то процесс создания экземпляров класса `Cat` совпадает с процессом создания экземпляров класса `Animal`. По этой же причине экземпляры дочернего класса, как и экземпляры родительского класса, имеют атрибуты `name` и `age`.

Определение и переопределение методов

Конечно, на практике дочерние классы не бывают пустыми, в них могут быть определены как собственные новые методы, так и переопределены методы родительского класса. Под переопределением метода родительского класса подразумевается определение метода в дочернем классе, имеющего то же имя, что и метод родительского класса. Например, определив в классе `Cat` метод `sleep()`, мы переопределим одноименный родительский метод.

Приведенный ниже код:

```
class Animal:
    def __init__(self, name, age):
        self.name = name
        self.age = age

    def sleep(self):
        return f'{self.name} спит zZ'

class Cat(Animal):
    def sleep(self):
        return f'{self.name} очень крепко спит zZ'

animal = Animal('Роджер', 2)
cat = Cat('Кемаль', 1)

print(animal.sleep())
print(cat.sleep())
```

ВЫВОДИТ:

```
Роджер спит zZ
Кемаль очень крепко спит zZ
```

Как мы видим, для экземпляров класса `Cat` используется метод `sleep()`, определенный в дочернем классе, а не в родительском. Аналогичным образом мы можем переопределить метод `__init__()`, например, если при создании экземпляров класса хотим указать дополнительные данные.

Приведенный ниже код:

```
class Animal:
    def __init__(self, name, age):
        self.name = name
        self.age = age

    def sleep(self):
        return f'{self.name} спит zZ'

class Cat(Animal):
    def __init__(self, name, age, eyecolor):
        self.name = name
        self.age = age
        self.eyecolor = eyecolor

    def sleep(self):
        return f'{self.name} очень крепко спит zZ'

cat = Cat('Кемаль', 1, 'желтый')

print(cat.name)
print(cat.age)
print(cat.eyecolor)
```

ВЫВОДИТ:

```
Кемаль
1
желтый
```

В примере выше в инициализаторе класса `Cat` мы дублируем строки с добавлением атрибутов `name` и `age`, которые аналогичным образом добавляются в инициализаторе класса `Animal`. Во избежание дублирования кода лучшим решением будет воспользоваться инициализатором родительского класса в инициализаторе дочернего класса.

Приведенный ниже код:

```
class Animal:
    def __init__(self, name, age):
        self.name = name
        self.age = age

    def sleep(self):
        return f'{self.name} спит zZ'

class Cat(Animal):
    def __init__(self, name, age, eyecolor):
        Animal.__init__(self, name, age) # вызываем инициализатор
        # родительского класса
        self.eyecolor = eyecolor

    def sleep(self):
        return f'{self.name} очень крепко спит zZ'

cat = Cat('Кемаль', 1, 'желтый')

print(cat.name)
print(cat.age)
print(cat.eyecolor)
```

ВЫВОДИТ:

```
Кемаль
1
желтый
```

Помимо переопределения какого-либо родительского метода, мы также можем определить в дочернем классе дополнительный метод.

Приведенный ниже код:

```
class Animal:
    def __init__(self, name, age):
        self.name = name
        self.age = age

    def sleep(self):
        return f'{self.name} спит zZ'

class Cat(Animal):
    def __init__(self, name, age, eyecolor):
        Animal.__init__(self, name, age)          # вызываем инициализатор
родительского класса
        self.eyecolor = eyecolor

    def sleep(self):
        return f'{self.name} очень крепко спит zZ'

    def jump(self):
        return f'{self.name} прыгает!'

cat = Cat('Кемаль', 1, 'желтый')

print(cat.jump())
```

ВЫВОДИТ:

```
Кемаль прыгает!
```

Обратите внимание, что метод, определенный в дочернем классе, не доступен в родительском классе.

Приведенный ниже код:

```
class Animal:
    def __init__(self, name, age):
        self.name = name
        self.age = age

    def sleep(self):
        return f'{self.name} спит zZ'

class Cat(Animal):
    def __init__(self, name, age, eyecolor):
        Animal.__init__(self, name, age) # вызываем инициализатор
        # родительского класса
        self.eyecolor = eyecolor

    def sleep(self):
        return f'{self.name} очень крепко спит zZ'

    def jump(self):
        return f'{self.name} прыгает!'

animal = Animal('Поджер', 1)

print(animal.jump())
```

ВЫВОДИТ:

```
AttributeError: 'Animal' object has no attribute 'jump'
```

На рассмотренных выше примерах с переопределением методов и созданием дополнительных мы можем заметить, как происходит поиск метода при обращении к нему: сперва он ищется в исходном классе, а затем, если не находится, в родительском.

Наследование атрибутов

В определении наследования, данном ранее, было сказано, что дочерний класс приобретает все атрибуты и методы родительского класса. До этого момента мы рассматривали только наследование методов, однако аналогичным образом ведут себя и атрибуты. Дочерний класс может их как унаследовать, так и переопределить.

Приведенный ниже код:


```
class Animal:
    en_name = 'animal'
    ru_name = 'животное'

class Cat(Animal):
    # наследует атрибуты родительского
    # класса
    pass

class Dog(Animal):
    # переопределяет атрибуты родительского
    # класса
    en_name = 'dog'
    ru_name = 'собака'

print(Cat.en_name)
print(Cat.ru_name)
print(Dog.en_name)
print(Dog.ru_name)
```

ВЫВОДИТ:

```
animal
животное
dog
собака
```

В примере выше класс `Cat` наследует атрибуты `en_name` и `ru_name` родительского класса `Animal`, в то время как класс `Dog` их переопределяет. Важно понимать, что класс `Cat` использует именно атрибуты класса `Animal`, а не создает их копии. То есть если в классе `Animal` их значения изменятся, это отразится и на классе `Cat`. Класс `Dog`, в свою очередь, переопределяет эти атрибуты и не зависит от изменений их значений в родительском классе.

Приведенный ниже код:

```
class Animal:
    en_name = 'animal'
    ru_name = 'животное'

class Cat(Animal):
    pass

class Dog(Animal):
    en_name = 'dog'
    ru_name = 'собака'

Animal.en_name = 'Animal'
Animal.ru_name = 'Животное'

print(Cat.en_name)
print(Cat.ru_name)
print(Dog.en_name)
print(Dog.ru_name)
```

ВЫВОДИТ:

```
Animal
Животное
dog
собака
```

Неявное наследование, класс object

Когда мы только начинали знакомство с классами, мы могли обратить внимание, что создаваемые нами пустые классы на самом деле не полностью пусты и по умолчанию обладают определенным функционалом. Дело в том, что все классы в Python являются наследниками класса `object`.

Приведенный ниже код:

```
class MyClass:
    pass

print(dir(MyClass()))
print(dir(object()))
```

ВЫВОДИТ:

```
['__class__', '__delattr__', '__dict__', '__dir__', '__doc__', '__eq__',
 '__format__', '__ge__', '__getattr__', '__gt__', '__hash__', '__init__',
 '__init_subclass__', '__le__', '__lt__', '__module__', '__ne__', '__new__',
 '__reduce__', '__reduce_ex__', '__repr__', '__setattr__', '__sizeof__', '__str__',
 '__subclasshook__', '__weakref__']
['__class__', '__delattr__', '__dir__', '__doc__', '__eq__', '__format__',
 '__ge__', '__getattr__', '__gt__', '__hash__', '__init__',
 '__init_subclass__', '__le__', '__lt__', '__ne__', '__new__', '__reduce__',
 '__reduce_ex__', '__repr__', '__setattr__', '__sizeof__', '__str__',
 '__subclasshook__']
```

Как мы видим, списки атрибутов и методов экземпляров нашего пустого класса и класса `object` практически одинаковы, лишь с тем различием, что экземпляр класса `MyClass` имеет три дополнительных атрибута в виде `__dict__`, `__module__` и `__weakref__`.



Так как любой класс в Python неявно наследуется от класса `object`, определения классов `class MyClass(object)` и `class MyClass` полностью равнозначны.

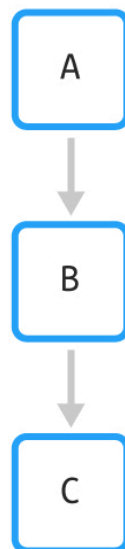
Многоуровневое наследование

Факт неявного наследования каждого класса от класса `object` позволяет сделать вывод, что с помощью наследования мы можем выстраивать иерархии классов, то есть, например, класс `C` может быть наследником класса `B`, который, в свою очередь, является наследником класса `A`.

```
class A:
    pass

class B(A):
    pass

class C(B):
    pass
```



Многоуровневое
наследование

Такое наследование называется **многоуровневым**. В данном случае класс **C** унаследует как атрибуты и методы класса **B**, так и класса **A**.

Приведенный ниже код:

```
class A:
    def method_A(self):
        print('Метод класса A')

class B(A):
    def method_B(self):
        print('Метод класса B')

class C(B):
    pass

c = C()

c.method_A()
c.method_B()
```

ВЫВОДИТ:

Метод класса A
Метод класса B

Однако нужно иметь в виду, что если класс B переопределит некоторые методы класса A, то классу C будут предоставлены именно переопределенные их версии.

Приведенный ниже код:

```
class A:
    def method(self):
        print('Метод класса A')

class B(A):
    def method(self):
        print('Метод класса B')

class C(B):
    pass

c = C()

c.method()
```

ВЫВОДИТ:

Метод класса B

Здесь мы уже можем окончательно понять, что при обращении к методу сперва он ищется в исходном классе, а затем во всей иерархии классов снизу вверх. Например, при обращении к методу класса C цепочка поиска будет следующей:

C --> B --> A --> object

Не забывайте, что все классы являются наследниками класса object, поэтому он обязательно присутствует в любой иерархии.

Примечания

Примечание 1. Атрибут класса __base__ позволяет получить родительский класс текущего класса.

Приведенный ниже код:

```
class A:
    pass

class B(A):
    pass

class C(B):
    pass

print(A.__base__)
print(B.__base__)
print(C.__base__)
```

ВЫВОДИТ:

```
<class 'object'>
<class '__main__.A'>
<class '__main__.B'>
```

Примечание 2. Функция `issubclass()` позволяет проверить, является ли класс прямым или косвенным наследником другого класса.

Приведенный ниже код:

```
class A:
    pass

class B(A):
    pass

class C(B):
    pass

print(issubclass(A, A))
print(issubclass(A, C))
print(issubclass(B, A))
print(issubclass(B, C))
print(issubclass(C, A))
print(issubclass(C, B))
```

ВЫВОДИТ:

```
True
False
True
False
True
True
```

Обратите внимание, что любой класс является наследником самого себя.

Примечание 3. Функция `isinstance()` позволяет проверить, является ли объект прямым или косвенным экземпляром некоторого класса.

Приведенный ниже код:

```
class A:  
    pass  
  
class B(A):  
    pass  
  
class C(B):
```



